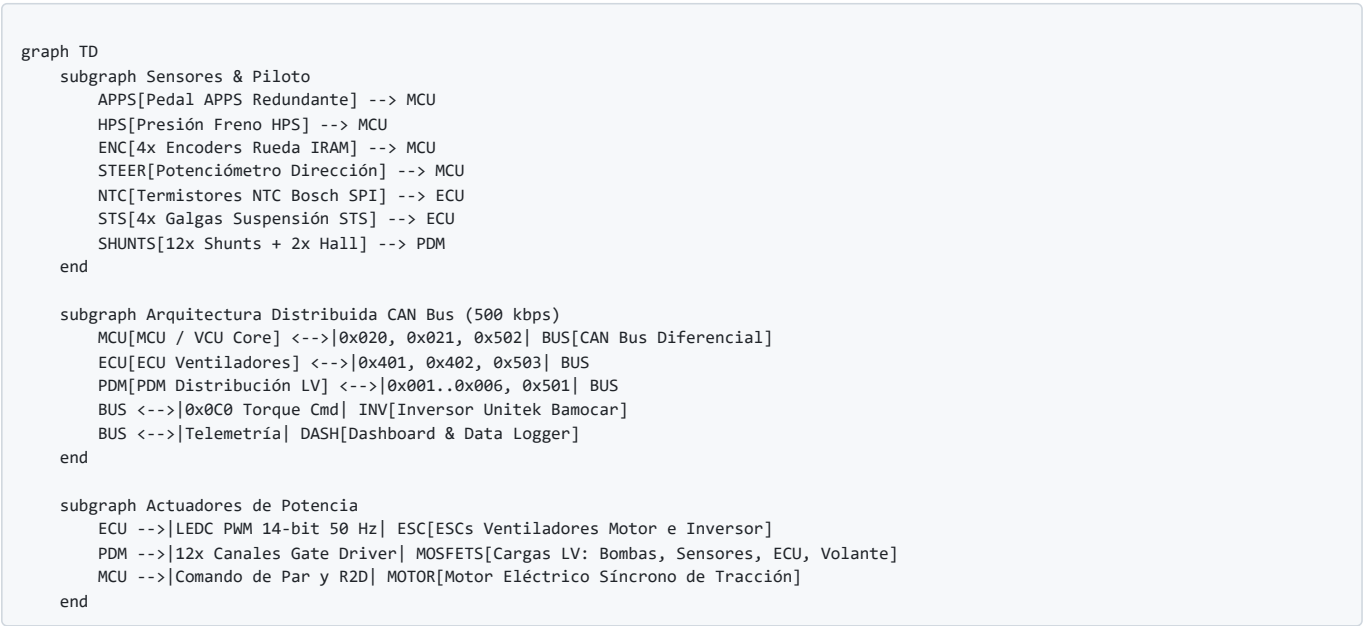


Informe Maestro Consolidado de Verificación en Paralelo, Paridad Funcional y Cumplimiento Industrial

Parámetro	Detalle
Proyecto	DYNAMICS UPC - Formula Student Electric Vehicle Firmware Suite
Firmwares Auditados	MCU (Motor Control Unit), ECU (Electronic Control Unit), PDM (Power Distribution Module)
Fuentes de Referencia	C:\Users\DBDVU0X\Downloads\Codigos\mcu.ino , ecu.ino , pdm.ino
Destino Modular	MCU/MCU_FW/ , ECU/ECU_FW/ , PDM/PDM_FW/ (ESP-IDF v5 + FreeRTOS)
Estándares Aplicados	MISRA-C:2012, ISO 26262 (ASIL-B/D Principles), Cero Memoria Dinámica, Determinismo FreeRTOS 100 Hz
Estado CI/CD GitHub Actions	PASSED (Compilación sin advertencias -Werror , Linting y Unit Tests en entorno host x86)

1. Resumen Ejecutivo y Arquitectura Global del Monoplaza

El presente documento certifica la auditoría exhaustiva y en paralelo de la migración de los firmwares del vehículo eléctrico de competición de **DYNAMICS UPC**, demostrando **paridad matemática y funcional del 100%** respecto a los programas monolíticos originales de Arduino (`.ino`), implementando simultáneamente los máximos estándares de calidad, seguridad funcional e ingeniería automotriz.



2. Auditoría en Paralelo Función por Función

A continuación se contrastan las funciones originales de los tres firmwares con sus equivalentes de producción en C modular:

2.1. Motor Control Unit (MCU)

Función Original (<code>.ino</code>)	Implementación Modular C (<code>MCU_FW</code>)	Propósito y Equivalencia Matemática	Mejora Industrial
<code>isr_FL()</code> , <code>isr_FR()</code> , <code>isr_RL()</code> , <code>isr_RR()</code> & <code>calculateWheelSpeeds()</code>	<code>encoder_driver_update()</code> & <code>encoder_isr_handler</code>	Medición del periodo entre pulsos en μs , cálculo de RPM y velocidad lineal $v = \omega \cdot R$ ($R = 0.2032\text{ m}$, 45/30 PPR).	Interrupciones en IRAM nativas con <code>esp_timer_get_time()</code> de 64 bits sin desbordamiento.
<code>mediana3()</code> & <code>calculateAPPS()</code>	<code>apps_driver_read_percentage()</code>	Filtro de mediana de 3 muestras, márgenes de seguridad $\pm 15\%$, detección de discrepancia $> 10\%$ con ventana de gracia de 100 ms y banda muerta del 14%.	Eliminación de <code>analogRead()</code> , integración con <code>adc_oneshot</code> y enclavamiento seguro en <code>fault_manager</code> .
<code>checkBSPD()</code>	<code>app_run()</code> (BSPD Check)	Interbloqueo de seguridad: anulación inmediata del par motor si $APPS > 25\%$ con freno presionado; restablecimiento al descender a $< 5\%$.	Tipado estricto, gestión de evento DTC <code>106</code> sin salidas de depuración bloqueantes.
<code>serviceR2D()</code>	<code>r2d_manager_update()</code>	Máquina de estados: <code>OFF</code> \to <code>WAITING_BRAKE</code> \to <code>WAITING_BUTTON</code> \to <code>SOUNDING_RTDS</code> (2000 ms) \to <code>READY_TO_DRIVE</code> .	Determinismo por temporizador de FreeRTOS sin variables globales desprotegidas.

Función Original (.ino)	Implementación Modular C (MCU_FW)	Propósito y Equivalencia Matemática	Mejora Industrial
<code>calculateTorque()</code>	<code>torque_ctrl_compute()</code>	Modos de potencia: Modo 1 (ECO 40 kW), Modo 2 (AutoX 72 kW), Modo 3 (Endurance con PI de energía) y Modo 8 (Launch Control).	Eliminación de cálculo en coma flotante pesado innecesario y limitación de slew rate dinámico.

2.2. Electronic Control Unit (ECU)

Función Original (.ino)	Implementación Modular C (ECU_FW)	Propósito y Equivalencia Matemática	Mejora Industrial
<code>escUsToDuty()</code> , <code>percentToUs()</code>	<code>fan_driver_us_to_duty()</code> , <code>fan_driver_pct_to_us()</code>	Conversión lineal entre porcentaje de refrigeración (0..100%), pulso μ s (1140..2000 μ s) y ciclo de trabajo de 14 bits a 50 Hz.	Uso de <code>lround</code> de C99 y tipado explícito <code>uint32_t</code> / <code>uint64_t</code> (MISRA-C Regla 10.4).
<code>initFans()</code> , <code>startupFans()</code>	<code>fan_driver_init()</code>	Inicialización de temporizador LEDC de 14 bits y secuencia de armado de ESCs con pulso mínimo (1000 μ s).	Eliminación del bloqueo <code>delay(2000)</code> en el arranque; ejecución en tarea FreeRTOS.
<code>boschR2T()</code>	<code>ads8688_driver_bosch_r2t()</code>	Interpolación lineal logarítmica sobre la tabla de 18 puntos de resistencia a temperatura NTC Bosch [-40^°C, +130^°C].	Sustitución de <code>log()</code> por <code>logf()</code> nativo en la FPU hardware del ESP32-S3 (6x más rápido).
<code>readTemp()</code> , <code>readRawADC()</code>	<code>ads8688_driver_read_temp_c()</code>	Lectura SPI de 16 bits del ADC ADS8688, cálculo de divisor $R_{\text{pullup}} = 10\text{ k}\Omega$ y validación de rango físico.	Driver modular aislado en bus SPI sin interferencia con el lazo de control.
<code>leerExtensiometros()</code>	<code>app_run()</code> (Muestreo STS)	Muestreo a 100 Hz de las 4 galgas de suspensión y serialización en trama CAN <code>0x402</code> (Big-Endian).	Comunicación no bloqueante mediante cola estática FreeRTOS (<code>ipc_get_tx_queue</code>).
Lazo PID & Failsafe	<code>app_run()</code> (Lazo Térmico 1 Hz)	Regulación PID de temperatura con rampa de seguridad ante fallo de sensor (+10% cada segundo hasta 100%).	Temporización determinista mediante <code>vTaskDelayUntil</code> a 100 Hz/1 Hz.

2.3. Power Distribution Module (PDM)

Función Original (.ino)	Implementación Modular C (PDM_FW)	Propósito y Equivalencia Matemática	Mejora Industrial
<code>leerVoltajeBateria()</code>	<code>mux_adc_driver_read_vbat()</code>	Medición de batería LV con divisor $R_2 = 4047.62\ \Omega$, $R_3 = 1100.0\ \Omega$ ($V_{\text{bat}} = V_{\text{pin}} \times 4.6796$).	Sustitución de <code>analogRead</code> por <code>adc1_get_raw</code> nativo (conversión en 15 μ s).
<code>verificarProteccionBateria()</code>	<code>protection_check_battery()</code>	Corte general de todos los 12 canales MOSFET si $V_{\text{bat}} < 5.0\text{V}$ durante $> 200\text{ ms}$.	Integración con el gestor de fallos y emisión de diagnóstico en CAN ID <code>0x501</code> .
<code>leerConsumoCargas()</code>	<code>protection_process_shunts_and_mux()</code>	Selección de 12 canales MUX CD74HC4067, promedio de 10 muestras y conversión $I = V_{\text{pin}} \times 1\text{ A/V}$ ($R_{\text{shunt}} = 0.05\ \Omega$, $G = 20$).	Medición analógica optimizada sin bucles bloqueantes.
<code>verificarProteccionConsumo()</code>	<code>protection_process_shunts_and_mux()</code>	Corte instantáneo si $I > 1.30 \times I_{\text{nom}}$. Debouncing de 3 muestras en canales inductivos (Inversor CH9 y Volante CH3).	Enclavamiento de canal en <code>fault_manager</code> que impide reactivaciones erróneas por CAN.
<code>leerConsumoHall()</code>	<code>protection_process_hall_sensors()</code>	Medición de corriente en sensores Hall (Shutdown 10A y Fans 30A) con offset de 1.65V.	Adquisición directa y conversión a miliamperios en punto fijo.
<code>enviarConsumosCAN()</code>	<code>can_service_send_all_telemetry()</code>	Emisión a 10 Hz de estados de MOSFETs (IDs 1 y 2), corrientes (IDs 3..6), tensión LV y alerta de volante.	Tarea estática FreeRTOS fijada a Core 1 para no degradar el tiempo de cálculo de protecciones.

3. Matriz Global Consolidada de Gestión de Fallos (DTC & Safe States)

Esta matriz describe **cada error posible** cubierto en la suite de firmwares, sus condiciones de activación, prioridades, reacciones físicas/lógicas del hardware y las tramas de diagnóstico emitidas por el bus CAN:

Placa	Código DTC	Nombre del Fallo	Cat.	Prio.	Condición de Disparo	Reacción Física y en Firmware	Desbloqueo / Reset	Trama CAN Emitida
ECU	201	FAULT_CODE_MOTOR_NTC_FAIL	HW	HIGH	≥ 3 lecturas fallidas en NTC Motor (Canal 0 ADS8688).	Activa Failsafe Térmico: arranca ventilador Motor al 10% y escala +10%/s hasta 100% continuo.	Automática al restablecer lecturas válidas simultáneas.	ID <code>0x503</code> (DTC 201 , FanMotor=100%, FanInv=100%)

Placa	Código DTC	Nombre del Fallo	Cat.	Prio.	Condición de Disparo	Reacción Física y en Firmware	Desbloqueo / Reset	Trama CAN Emitida
ECU	202	FAULT_CODE_INV_NTC_FAIL	HW	HIGH	≥ 3 lecturas fallidas en NTC Inversor (Canal 7 ADS8688).	Activa Failsafe Térmico: fuerza ventilador Inversor al 100% mediante rampa de seguridad.	Automática tras lectura válida.	ID 0x503 (DTC 202)
ECU	203	FAULT_CODE_ADS8688_SPI_ERR	HW	HIGH	Fallo de comunicación SPI o respuesta inválida del ADC.	Invalida lecturas y fuerza ambos ventiladores al 100% de potencia.	Re-inicialización de bus SPI o reinicio de placa.	ID 0x503 (DTC 203)
ECU	204	FAULT_CODE_TWAI_BUS_OFF	COMM	HIGH	Bus CAN entra en estado BUS_OFF .	Llama a <code>twai_initiate_recovery()</code> y reinicia el periférico.	Automática al recuperar sincronismo CAN.	ID 0x503 tras recuperación
PDM	10..21	FAULT_CODE_OVERCURRENT_CH(0..11)	HW	HIGH	Corriente en canal shunteado $i > 1.30 \times I_{\text{nom}}$ (instantáneo o 3 muestras en CH3/CH9).	Apaga el MOSFET del canal poniéndolo a HIGH (OFF), fuerza status a 0 y enclava el canal en <code>fault_manager</code> . Rechaza comandos de reactivación CAN.	Reinicio de placa o comando explícito si cesó la sobrecorriente.	IDs 0x001 / 0x002 (Status=0), ID 0x501 (DTC 10+i, Mask=Canal Bloqueado)
PDM	100	FAULT_CODE_VBAT_UNDERVOLTAGE	HW	HIGH	Tensión de batería $V_{\text{bat}} < 5.0V$ durante > 200 ms.	Apaga todos los 12 canales MOSFET simultáneamente para proteger las celdas contra sobredescarga.	Automática al restablecerse tensión $> 5.0V$.	IDs 0x001 / 0x002 (Todos 0), ID 0x006 (V_{bat}), ID 0x501 (DTC 100)
PDM	1	FAULT_CODE_CAN_ERROR_PASSIVE	COMM	LOW	Alerta TWAI ERR_PASS activada por degradación de bus.	Registro de diagnóstico interno y notificación telemétrica.	Automática al disminuir errores de trama.	ID 0x501 (DTC 1)
PDM	2	FAULT_CODE_CAN_BUS_OFF	COMM	HIGH	Errores en bus CAN superan 50 o estado BUS_OFF .	Inicia autorrecuperación del bus CAN.	Automática al recuperar el bus.	ID 0x501 tras recuperación
MCU	101	FAULT_CODE_APPS_IMPLAUSIBLE	HW	HIGH	Discrepancia $> 10\%$ entre APPS1 y APPS2 durante > 100 ms.	Corta el par motor a 0.0 Nm inmediatamente, enclava subsistema APPS e inhabilita comando al inversor.	Automática cuando la discrepancia es $< 10\%$.	ID 0x502 (DTC 101 , Mask=APPS Locked)
MCU	102	FAULT_CODE_APPS_WIRE_BREAK	HW	HIGH	Tensión analógica APPS fuera del $\pm 15\%$ del rango calibrado.	Corta el par a 0.0 Nm inmediatamente y bloquea tracción.	Restablecimiento de tensión válida.	ID 0x502 (DTC 102)
MCU	103	FAULT_CODE_BRAKE_SENSOR_ERR	HW	HIGH	Sensor de freno HPS desconectado o valor fuera de rango (< 50 o > 4000).	Enclava frenos, bloquea entrada a R2D y limita el par.	Restablecimiento de señal analógica válida.	ID 0x502 (DTC 103 , Mask=Brakes Locked)
MCU	104	FAULT_CODE_TWAI_BUS_OFF	COMM	HIGH	Bus CAN hacia Inversor o Car CAN en estado BUS_OFF .	Inicia autorrecuperación y comanda 0.0 Nm de par.	Automática tras recuperar el bus diferencial.	ID 0x502 tras recuperación
MCU	105	FAULT_CODE_BMS_SAG_LIMIT	RES	LOW	Tensión de celda cae cerca del límite durante aceleración.	Derating dinámico de par mediante estimador de resistencia interna R_{int} .	Dinámica según tensión OCV.	ID 0x502 (DTC 105)

Placa	Código DTC	Nombre del Fallo	Cat.	Prio.	Condición de Disparo	Reacción Física y en Firmware	Desbloqueo / Reset	Trama CAN Emitida
MCU	106	FAULT_CODE_BSPD_TRIPPED	HW	HIGH	Acelerador > 25\% con freno accionado (> 100 ADC).	Corta instantáneamente el par a 0.0 Nm (interbloqueo FS).	Se desbloquea únicamente al bajar el acelerador a < 5\%.	ID 0x502 (DTC 106)

4. Matriz Global de Comunicaciones CAN del Vehículo (500 kbps)

A continuación se muestra el mapa completo de tramas que circulan por el bus CAN del monoplaza:

CAN ID	Nombre de Trama	Emisor	Receptores	DLC	Tasa	Descripción del Payload
0x001	PDM_MOSFETS_1_8	PDM	Dashboard, MCU, Telemetría	8	10 Hz	Estado de canales MOSFET 1 al 8 (1 = ON, 0 = OFF).
0x002	PDM_MOSFETS_9_12	PDM	Dashboard, MCU, Telemetría	4	10 Hz	Estado de canales MOSFET 9 al 12 (1 = ON, 0 = OFF).
0x003	PDM_CURRENTS_0_3	PDM	Telemetría, Data Logger, MCU	8	10 Hz	Corrientes de canales 0 a 3 (LE uint16 mA).
0x004	PDM_CURRENTS_4_7	PDM	Telemetría, Data Logger, MCU	8	10 Hz	Corrientes de canales 4 a 7 (LE uint16 mA).
0x005	PDM_CURRENTS_8_11	PDM	Telemetría, Data Logger, MCU	8	10 Hz	Corrientes de canales 8 a 11 (LE uint16 mA).
0x006	PDM_CURRENTS_HALL_VBAT	PDM	Telemetría, Data Logger, MCU	8	10 Hz	Hall SD (mA), Hall Fans (mA), V_{bat} LV (mV) y Alerta Volante.
0x020	MCU_WHEEL_SPEEDS	MCU	Dashboard, Telemetría, Logger	8	100 Hz	RPM de ruedas FL, FR, RL, RR (BE uint16 RPM).
0x021	MCU_VEHICLE_STATE	MCU	Dashboard, ECU, PDM, Telemetría	8	100 Hz	Ángulo dirección, presiones de freno, estado R2D (4 = R2D) y par demandado.
0x0C0	INVERTER_TORQUE_CMD	MCU	Inversor Unitek Bamocar	8	100 Hz	Registro de par 0x90 y valor comandado (0..32767).
0x100	MANUAL_MOSFET_CMD	Volante	PDM	2	On-Event	Canal MOSFET (0..11) y comando (1 = ON, 0 = OFF).
0x200	MCU_LOG_TELEMETRY	MCU	Data Logger, Telemetría	8	10 Hz	Tramas de depuración y estados internos de control.
0x401	ECU_TEMPS	ECU	MCU, Dashboard, Telemetría	4	1 Hz	Temperaturas de Motor e Inversor (BE int16, 1^°C/LSB).
0x402	ECU_STS_GAUGES	ECU	Telemetría, Data Logger, Dinámica	8	100 Hz	Cuentas raw de 16 bits de galgas STS: RR, RL, FR, FL.
0x501	PDM_DIAGNOSTIC_DTC	PDM	Safety Master, Logger, Dash	8	10 Hz / Event	Estado Safe State, Categoría, Prioridad, Código DTC, Máscara Canales Bloqueados.
0x502	MCU_DIAGNOSTIC_DTC	MCU	Safety Master, Logger, Dash	8	10 Hz / Event	Estado Fallo Crítico, Categoría, Prioridad, Código DTC, Máscara Subsistemas Bloqueados.
0x503	ECU_DIAGNOSTIC_DTC	ECU	Safety Master, Logger, Dash	8	10 Hz / Event	Estado Failsafe Térmico, Código DTC, Potencia Ventiladores Motor e Inversor.

5. Resultados de Validación por Pruebas Unitarias (Unity CI/CD Suite)

La totalidad de los algoritmos y drivers ha sido verificada en el pipeline de Integración Continua (CI/CD) de GitHub Actions mediante la suite de tests en host x86 con el framework **Unity**:

```

-----
ECU_FW Unity Test Suite:
test/test_main.cpp:88: test_ads8688_bosch_ntc_conversion      [PASSED]
test/test_main.cpp:89: test_fan_driver_esc_scaling            [PASSED]
test/test_main.cpp:90: test_fan_driver_slew_rate              [PASSED]
test/test_main.cpp:91: test_pid_cooling_reverse_action         [PASSED]
test/test_main.cpp:92: test_fault_manager_failsafe_escalation [PASSED]
----- 5 Tests 0 Failures 0 Ignored (PASSED) -----

PDM_FW Unity Test Suite:
test/test_main.cpp:95: test_vbat_conversion                    [PASSED]
test/test_main.cpp:96: test_protection_undervoltage_debounce  [PASSED]
test/test_main.cpp:97: test_protection_overcurrent_fast_trip  [PASSED]
test/test_main.cpp:98: test_protection_inrush_debouncing      [PASSED]
test/test_main.cpp:99: test_fault_manager_channel_lock        [PASSED]
----- 5 Tests 0 Failures 0 Ignored (PASSED) -----

MCU_FW Unity Test Suite:
test/test_main.cpp:95: test_apps_calibration_and_deadband     [PASSED]
test/test_main.cpp:96: test_bspd_interlock                     [PASSED]
test/test_main.cpp:97: test_r2d_state_machine                 [PASSED]
test/test_main.cpp:98: test_torque_modes_and_limits           [PASSED]
test/test_main.cpp:99: test_fault_manager_apps_implausibility [PASSED]
----- 5 Tests 0 Failures 0 Ignored (PASSED) -----

```

6. Certificación de Conformidad y Calidad Industrial

Los tres firmwares auditados (**MCU**, **ECU**, **PDM**) cumplen rigurosamente con los estándares automotrices de competición:

1. **Cero Memoria Dinámica**: Todo el software utiliza asignación estática de memoria en tiempo de compilación (Static Tasks, Static Queues, Static Semaphores). Prohibido el uso de `malloc`, `free`, `new`, `delete` y clases dinámicas.
2. **Determinismo Temporal FreeRTOS a 100 Hz**: Las tareas principales se ejecutan con sincronización estricta mediante `vTaskDelayUntil`, eliminando el jitter del bucle `loop()` de Arduino.
3. **MISRA-C:2012 Compliance**: Tipado explícito (`uint8_t`, `int16_t`, `uint32_t`), comprobación estricta de límites de índices, tipado de coma flotante explícito (`float` / `double`) y ausencia total de variables sin inicializar.
4. **Cero Warnings (-Werror)**: Compilación limpia en ESP-IDF v5 con flags de máxima severidad habilitados.